Lab 06 — Least privilege, logs, and a restore

Course: Maxim Nyansa Cybersecurity Bootcamp, Ghana
Trainee: James Ackah Blay · Aiyinasi, Ghana
Week: 6 — Database security
Estimated time: 5 hours
Difficulty: Intermediate. You will run a database you can break and then restore, on purpose, on a copy.

> **Tutor note.** Use PostgreSQL on the Ubuntu lab VM or on the laptop. Do not point these commands at a school or hosting server. The detection task reads a synthetic log. It does not send queries to a server.

1. Learning objectives

Objective	Job skill
Separate an application login from an administrator	Least privilege on a data store
Explain why values must be parameters	Talk about injection as a failure of query construction, and name the fix
Flag a bad statement in a log	Database monitoring, which ends up in the SIEM
Restore from a dump into a copy	Prove a backup before you need it

2. Prerequisites and setup

Week 5's idea of "input is data" still applies. PostgreSQL 14 or newer. On Ubuntu:

```
        sudo apt-get update
        sudo apt-get install -y postgresql
        sudo systemctl enable --now postgresql
```

You will use the local `postgres` account through `sudo -u postgres`. That is an administrator. The application role you create must not be that account.

3. Scenario

Palm Court's order book is now a database: customers, orders, and a role for the counter app. Kwame wants the app to connect as `postgres` "so nothing breaks". You will refuse that, build a smaller role, take a dump, break a copy, and restore the copy.

4. Tasks

Task 1 — Create the lab database

```
        sudo -u postgres psql -c "CREATE DATABASE palmcourt;"
        sudo -u postgres psql -d palmcourt -c "CREATE TABLE orders (id int PRIMARY KEY, item
text NOT NULL);"
        sudo -u postgres psql -d palmcourt -c "INSERT INTO orders VALUES (1, 'notebook');"
```

Task 2 — Least privilege

```
        sudo -u postgres psql -d palmcourt -c "CREATE ROLE counter LOGIN PASSWORD 'replace-
this-local-password';"
        sudo -u postgres psql -d palmcourt -c "GRANT SELECT, INSERT ON orders TO counter;"
        sudo -u postgres psql -d palmcourt -c "\\du"
```

Choose a password you can throw away. Do not commit it. Confirm `counter` cannot drop the table:

```
PGPASSWORD='replace-this-local-password' psql -h 127.0.0.1 -U counter -d palmcourt -c "DROP TABLE orders;"
```

> **You should now see** an error from PostgreSQL denying the drop. If the drop succeeds, `counter` is too strong. Revoke the extra privilege and try again. `\\du` should show `counter` without superuser.

Compare this with `../../role-permission-matrix/permission-matrix.csv`. The Python demo in `../database-security-checklist/` is the same idea on SQLite:

```
python3 -m pytest ../database-security-checklist/test_db_access.py
```

Task 3 — Read a log, do not attack the server

`samples/db-statements.log` is synthetic.

```
python3 sql_log_review.py samples/db-statements.log
```

One line is an ordinary insert. One line is a statement that does not look like the application. Write what you would ask a developer: show me the query in the code, and show me that the user value is a parameter. The Week 6 checklist calls that control D-01.

Do not type the flagged statement into `psql` against any database, including this one. The lab is the log.

Task 4 — Dump and restore a copy

```
sudo -u postgres pg_dump palmcourt > "$HOME/palmcourt.dump"
sudo -u postgres psql -c "CREATE DATABASE palmcourt_restore;"
sudo -u postgres psql -d palmcourt_restore -f "$HOME/palmcourt.dump"
sudo -u postgres psql -d palmcourt_restore -c "SELECT id, item FROM orders;"
```

Keep the dump out of git. It is a practice file, and a later one might not be.

> **You should now see** the notebook row in `palmcourt_restore`. Then drop only the restore database if you want the VM tidy:
>
> ```
> sudo -u postgres psql -c "DROP DATABASE palmcourt_restore;"
> ```
>
> Do not drop `palmcourt` until you have decided the exercise is finished.

Task 5 — Fill the recovery plan

In `../role-permission-matrix/recovery-plan.md`, write the commands you ran and whether the row matched. If PostgreSQL is not installed yet, leave the commands blank and say so.

5. Troubleshooting

What you see	What it means
`Peer authentication failed`	You used `-h 127.0.0.1` without a password, or the opposite. Local sockets use peer auth. TCP uses the password
`role "counter" already exists`	You ran Task 2 twice. Continue, or drop the role and recreate it
Dump file is empty	`pg_dump` failed and the shell still created the file. Read the error above the prompt
Detector flags the ordinary insert	Look at whether your pattern is too wide. Record it. The sample test expects one flag

6. Stretch task

Add a second role, `reporter`, with `SELECT` only. Show the grant command and a denied `INSERT`. Update the permission matrix if you want the portfolio to match the database. Do not grant `reporter` the ability to read every database on the server.

7. Evidence

- `\\du` output.
- The denied `DROP` or `INSERT`.
- Detector output labelled synthetic.
- `SELECT` from the restored copy, or a note that PostgreSQL is not installed.

8. Reflection

1. Why is `postgres` the wrong login for the counter app?
2. What did the restore prove that the dump file alone did not?
3. What personal data is not in this lab database, and what would change if a real phone number were?

9. Interview questions

1. What is least privilege on a database role?
2. Why do parameterised queries matter?
3. How do you test a logical backup?
4. What would you look for in a database log before you call it an incident?

10. Grading rubric

Criterion	Exceeds	Meets	Not yet
Roles	A second read-only role is demonstrated	`counter` cannot drop the table	The app uses a superuser
Detection	You explain the flagged line and the developer question	You ran the script on the sample	You replayed the statement against a server
Restore	The row is shown in the copy	The dump exists and you say it is untested	No note
Secrets	Passwords stayed out of git	You used a local throwaway password	A password committed